Self-Hosted Cloud File Storage — Database Design Document

Status: v6 — final (post five design-review passes; all previously open items resolved) **Scope:** Data model / database design only. Application architecture, API design, and infrastructure deployment are out of scope for this document.

---

1. Purpose and Goals

This document describes the database design for a self-hosted, multi-user cloud file storage system — functionally similar to a self-hosted Dropbox or Nextcloud. It captures every entity, field, constraint, and design decision made across the initial design and five subsequent adversarial review passes, along with the reasoning behind each fix.

1.1 Functional goals

- Multiple user accounts, each with their own private storage space.
- Hierarchical folder organization (nested folders, arbitrary depth).
- File upload, download, rename, move, and delete.
- Sharing: both direct per-user sharing (grant a specific person access) and public share links (anyone with the link/token).
- Per-user storage quotas, enforced on upload.
- An administrative "super-user" role that can view all files and users, see system-wide and per-user storage statistics, view usage-frequency data, and alter other users' permissions and quotas.
- Favorites/starred items, personal tags/labels, and generated thumbnails/previews for images, video, and PDFs.

1.2 Explicit non-goals

- **No file version history.** Overwriting a file replaces it; there is no "restore previous version" feature. This was an explicit scope decision, not an oversight — it significantly simplifies the FILES table and avoids an entire versioning subsystem.
- **No real-time collaborative editing** (e.g., simultaneous multi-cursor editing like Google Docs). This is a file store, not a document editor.
- **No end-to-end encryption** is modeled at the database layer. If required, it would be an application-layer concern (encrypt before upload) and is not reflected in this schema.
- **No team/group accounts.** Sharing is exclusively user-to-user (PERMISSIONS grantee is always an individual `user_id`) or via public SHARE_LINKS — there is no concept of a group with members that can be granted access as a single unit. **Confirmed out of scope by the system owner.** This also

means TAGS' per-user scoping (§3.12) is not expected to need revisiting for a shared/team tag taxonomy.

1.3 Technology assumptions

- **Metadata store:** PostgreSQL. Chosen for strong relational integrity guarantees, partial/expression indexes, `CHECK` constraints, deferrable constraints, and native support for `jsonb` (used in the activity log).
- **Blob storage:** An S3-compatible object store (e.g., MinIO), self-hosted alongside the database. The database never stores file bytes — only a reference (`storage_key`) to the object in the store.
- **Deployment shape:** Single self-hosted server (or small cluster) — not a large multi-tenant SaaS. Several design decisions below (e.g., accepting pure application-layer authorization with no database-level backstop) are justified specifically by this deployment shape and would need revisiting at larger scale or in a less-trusted operating environment.

---

2. Design Process

This schema was not designed in a single pass. It went through six iterations:

1. **Initial design** — the baseline entity set based on functional requirements (users, files, folders, sharing, activity).
2. **Review pass 1** — stress-tested the initial design against real implementation scenarios (concurrency, cascading deletes, orphaned data, missing indexes). Found 9 issues.
3. **Review pass 2** — cross-verified that the pass-1 fixes actually worked as designed, rather than just looking correct on paper. Found that several fixes were incomplete (e.g., a cache invalidation strategy that didn't specify what invalidated it). Found 7 issues.
4. **Review pass 3** — introduced the admin/super-user feature, then reviewed the *consequences* of that feature against the already-hardened schema (e.g., does admin editing permissions conflict with the "exactly one owner" constraint added in pass 2?). Found 6 issues, several of which were genuine new bugs introduced by the admin feature rather than pre-existing gaps.
5. **Review pass 4** — introduced three new features (favorites, tags, thumbnails) on top of the now-hardened schema, then reviewed those additions for consistency with every pattern established in passes 1–3 (the polymorphic reference pattern, the DELETION_QUEUE cascade responsibility, the ownership-reassignment-on-user-deletion rule). Found 6 issues, including one genuine race condition and one case where a previously-established pattern (PERMISSIONS ownership reassignment) was wrongly assumed to apply to a new entity (TAGS) where it didn't.
6. **Review pass 5 (final)** — after the system owner made the last six open decisions (cycle-check locking strategy, tag scope, team-account scope, trash retention window, admin audit sufficiency, deduplication deferral), this pass specifically checked whether *finalizing* those decisions exposed anything that had been masked by "TBD," rather than re-reviewing anything already settled. Found 4 concrete issues (two of them in the just-decided advisory-lock approach itself) and explicitly confirmed 2 of the six decisions introduced no new problems.

Each fix below is labeled with which pass introduced it, because the reasoning ("why does this field exist") is often more valuable than the field itself when this schema is eventually translated into SQL.

---

3. Entity Reference

3.1 USERS

The account table. One row per person who can log in.

Field	Type (conceptual)	Notes
`id`	UUID, PK	
`email`	string, unique	Login identifier.
`password_hash`	string	Never store plaintext; hash with a slow algorithm (bcrypt/ argon2) at the application layer.
`display_name`	string	
`role`	enum: `user`, `admin`	**(Pass 3 / admin feature)** Determines whether this account bypasses normal permission checks. See §4 for the full admin design.
`storage_quota_bytes`	bigint	Maximum bytes this user may store.
`storage_used_bytes`	bigint	Denormalized running total of bytes currently stored by this user. Updated on every upload and every completed deletion — never computed live via `SUM()`, for performance.
`created_at`, `updated_at`	timestamp	
`is_active`	boolean	Soft-disable an account without deleting it.

Constraints:

- A rule ensures `storage_used_bytes` cannot exceed `storage_quota_bytes`, but — critically — **this rule must only fire when `storage_used_bytes` increases** (i.e., on the upload path). It must **not** block an administrator from lowering a user's `storage_quota_bytes` below their current usage. *(Pass 3 fix — the original naive version of this rule as a plain cross-column `CHECK` would have made it impossible for an admin to cap an abusive user's quota after the fact, since the `UPDATE` on `storage_quota_bytes` would fail the moment the new quota was below current usage.)* The correct behavior: lowering quota below current usage is **allowed**; it simply blocks *new* uploads until the user frees space.
- **Implementation clarification:** because this rule depends on comparing the *new* `storage_used_bytes` against the row's *previous* value (to distinguish "this is an increase" from "this is a decrease" or "only the quota changed"), it **cannot be written as a plain `CHECK` constraint** — a

`CHECK` in Postgres only sees the row being written, not its prior state. This must be implemented as a `BEFORE UPDATE` trigger (or, more simply, enforced entirely at the application layer via the same atomic conditional-update pattern described in §5.3) that inspects `OLD.storage_used_bytes` vs. `NEW.storage_used_bytes` before deciding whether to reject the write.

- **Quota is editable only by an admin.** This is an application-layer rule, not database-enforced — a regular user cannot change their own `storage_quota_bytes` through any normal code path.

3.2 SESSIONS

(Added in Pass 1 — the original design had no authentication/session model at all, despite being a multi-user system with login.)

Tracks active login sessions / refresh tokens, so users can be logged in on multiple devices and sessions can be individually revoked.

Field	Type	Notes
`id`	UUID, PK	
`user_id`	FK → USERS	
`refresh_token_hash`	string, unique	Store a hash of the token, never the token itself.
`device_info`	string	User-agent / device name, for a "manage your devices" UI.
`expires_at`	timestamp	
`created_at`, `revoked_at`	timestamp	`revoked_at` nullable; set on logout or admin-forced revocation.

3.3 FOLDERS

Hierarchical containers. Self-referencing tree structure.

Field	Type	Notes
`id`	UUID, PK	
`name`	string	
`owner_id`	FK → USERS	
`parent_folder_id`	FK → FOLDERS, nullable	Null = root-level folder.
`materialized_path`	string	**(Pass 2)** A denormalized path (e.g. `/root-id/child-id/grandchild-id/`) that lets the system answer "is X an ancestor

Field	Type	Notes
		of Y" and "list this entire subtree" with an indexed prefix query instead of a recursive tree walk on every request.
`created_at`, `updated_at`	timestamp	
`deleted_at`	timestamp, nullable	Soft-delete marker (see §5.2, Soft delete and trash).

Constraints and behavioral rules:

- **Unique sibling names.** `UNIQUE(parent_folder_id, name) WHERE deleted_at IS NULL`. *(Pass 1 fix.)* This is a **partial** unique index specifically so that a trashed folder's name doesn't block a live folder from reusing it. Without this constraint, two concurrent "create folder" requests racing on the same name in the same parent would both succeed, producing silent duplicates.
- **materialized_path must be rewritten for the entire subtree on every move, in the same transaction as the move.** *(Pass 2 fix, refined in Pass 3.)* A naive implementation that only updates the moved folder's own path leaves every descendant's path stale, silently breaking any subtree query or permission-inheritance logic that relies on the path. Moving `/A/B/C` under `/X` must rewrite the path prefix on `C` **and every folder nested inside** `C`, as one atomic bulk update.
- **Cycle prevention on move.** A trigger (or equivalent application-transaction check) must reject any update to `parent_folder_id` that would make a folder its own ancestor. Two important refinements came out of review:
- **(Pass 2)** Checking only the *direct* parent is insufficient — it only catches the immediate case (`A` becomes child of `A`), not a folder being moved several levels down into its own descendant tree (`A` moved to be a child of its own grandchild `C`). The check must walk the **full ancestor chain** from the proposed new parent up to the root (or use the materialized_path prefix, once that exists) and reject if the moved folder's own id appears anywhere in that chain.
- **(Pass 3 — highest-risk item, now resolved)** This check has a concurrency hole: two unrelated concurrent move operations (e.g., moving `A` under `B` while moving `B` under `A`) can each read a stale ancestor chain and both pass validation before either commits, producing a cycle despite the trigger. **Decision (owner-confirmed): a Postgres advisory lock, keyed on the subtree root being moved, is taken before the ancestor-walk cycle check runs.** This was chosen over `SERIALIZABLE` isolation + retry (no isolation-level change or app-side retry logic required, simpler to reason about) and over row-level `FOR UPDATE` locking up the full ancestor chain (avoids the deadlock-ordering complexity of locking multiple rows across concurrent transactions) as the best fit for a self-hosted, low-concurrency deployment.
- **(Pass 5 — two corrections found while finalizing this decision)**
  1. **Must be `pg_advisory_xact_lock`, not the plain session-scoped `pg_advisory_lock`.** Session-scoped advisory locks are tied to the physical database connection, not the logical transaction. Any connection pooler running in transaction-pooling mode (e.g., PgBouncer in `transaction` mode — a common, often default, setup for a web app) can hand that same backend connection to a *different* request the moment the current transaction commits, and session-level locks are explicitly documented as unsafe under transaction-mode pooling. If the app crashes or a connection is returned to the pool before an explicit unlock, the lock leaks until the

underlying backend connection itself closes. `pg_advisory_xact_lock` avoids all of this — it automatically releases at transaction end (commit or rollback), matching the exact lifetime the move operation needs, with no explicit unlock call required.

2. **Locking only the moved node's own subtree root is not sufficient** to prevent the specific case of two folders being moved into each other's subtrees simultaneously (`A` moved into `B` while `B` is concurrently moved into `A`). Each transaction would acquire a *different* lock key (its own subtree root), so there's no mutual exclusion between them — both locks succeed, both ancestor-walks run before either commits, and neither walk can see the other's not-yet-committed move, so both can pass validation and a real cycle gets created. **The correct fix: lock both the moved node's subtree root and the target parent's ancestry root, in a canonical order (e.g., sorted by folder id) before running the ancestor walk.** This forces the second transaction to wait for the first to fully commit, at which point its ancestor walk correctly observes the completed first move.

3.4 FILES

Field	Type	Notes
`id`	UUID, PK	
`name`	string	
`owner_id`	FK → USERS	
`folder_id`	FK → FOLDERS, nullable	Null = file lives at the user's root.
`size_bytes`	bigint	
`mime_type`	string	
`checksum`	string	SHA-256 (or similar) of the file content. Useful for integrity verification and as a building block for future deduplication, even though dedup itself is out of scope.
`storage_key`	string	The object key/path in the S3-compatible blob store.
`upload_status`	enum	**(Pass 1, tied to UPLOAD_SESSIONS)** e.g. `pending`, `complete` — a file row can exist before its bytes are fully uploaded (multipart uploads), and the app needs to distinguish a real, downloadable file from one still being assembled.
`thumbnail_status`	enum: `none`, `pending`, `ready`, `failed`	**(Pass 4 / thumbnails feature)** Tracks the state of an asynchronously generated preview image. See §3.13 (THUMBNAIL_QUEUE) for the generation mechanism.
`thumbnail_key`	string, nullable	**(Pass 4)** The object key of the generated thumbnail in the blob store, once ready. Null until generation completes.
	timestamp	

Field	Type	Notes
`created_at`, `updated_at`		
`deleted_at`	timestamp, nullable	Soft-delete marker.

Constraints and behavioral rules:

- **Unique sibling names.** `UNIQUE(folder_id, name) WHERE deleted_at IS NULL` — same reasoning as FOLDERS above. *(Pass 1.)*
- **Moving a file to a different folder must invalidate any EFFECTIVE_PERMISSIONS rows it inherited from its old folder.** *(Pass 2.)* Without this, a user who had access only because the file used to live in a folder shared with them would silently retain that access after the file moves elsewhere.
- **`storage_key` must be non-guessable** — a random UUID or content-hash-derived key, **not** a predictable pattern like `{folder_id}/{filename}`. *(Pass 3.)* Combined with a MinIO bucket policy that denies anonymous/public reads, this ensures the application is the *only* path to a file's bytes; nothing about the key itself should let someone enumerate or guess other users' files.
- **`thumbnail_key` must follow the exact same non-guessability and bucket-policy rule as `storage_key`.** *(Pass 4.)* A thumbnail is still a rendering of potentially sensitive content — it should not get a weaker access-control story than the original file just because it's "only a preview."
- **A file move or rename does not invalidate or require regenerating the thumbnail** — thumbnails are keyed by `file_id`, not by path. *(Pass 4.)* Since this schema has no versioning, a content change is always a new FILES row (delete + re-upload), which naturally gets its own fresh thumbnail; there is no "same file, new content" case to handle.

3.5 UPLOAD_SESSIONS

(Added in Pass 2 — the original design had no support for uploading anything larger than a single request body.)

Tracks in-progress multipart/chunked uploads against the blob store.

Field	Type	Notes
`id`	UUID, PK	
`file_id`	FK → FILES, nullable	Nullable because the FILES row may not exist yet when the upload session starts.
`user_id`	FK → USERS	
`minio_upload_id`	string	The underlying object-store's multipart upload identifier.
`total_parts`, `parts_received`	integer	Progress tracking.

Field	Type	Notes
`status`	enum: `pending`, `complete`, `aborted`	
`expires_at`	timestamp	Abandoned upload sessions should expire and be cleaned up; otherwise they'd accumulate as orphaned partial uploads in the blob store forever.
`created_at`	timestamp	

Relevant to admin stats (§4): an in-progress upload session represents bytes that are neither fully "used" nor "free" from the user's quota perspective — see the caveat under §4.3.

3.6 PERMISSIONS

Direct, per-user access grants on a file or folder.

Field	Type	Notes
`id`	UUID, PK	
`item_type`	enum: `file`, `folder`	Part of a polymorphic reference — see §5 for a full discussion of this pattern.
`item_id`	UUID	The id of the file or folder, interpreted according to `item_type`. No real foreign key (Postgres cannot FK against "one of two tables"), so referential integrity here is an application/trigger responsibility.
`user_id`	FK → USERS	The grantee — who is being given access.
`role`	enum: `owner`, `editor`, `viewer`	
`granted_by`	FK → USERS	Who granted this access — for audit purposes.
`created_at`	timestamp	

Clarification on the initial `owner` row: when a file or folder is created, exactly one PERMISSIONS row must be inserted immediately, with `role = 'owner'`, `user_id` = the creator, and `granted_by` **= the creator as well (self-referential)** — this satisfies the "exactly one owner" invariant (below) from the moment the item exists, rather than leaving a window where a newly created item has zero PERMISSIONS rows. `granted_by` is never null; there is no code path that creates an item without simultaneously creating its owner grant.

Constraints and behavioral rules:

- `UNIQUE(item_type, item_id, user_id)` *(Pass 1)* — a user can only have one role per item; granting a new role should update the existing row, not create a duplicate.
- `UNIQUE(item_type, item_id) WHERE role = 'owner'` *(Pass 2)* — enforces that an item has **exactly one** owner at any time. Without this, nothing prevented an item from ending up with zero owners (e.g., if the owning user's account is deleted) or, in principle, multiple.
- **This owner-uniqueness constraint must be declared `DEFERRABLE INITIALLY DEFERRED`, and ownership transfer must go through one single, atomic operation** *(Pass 3)*. This was a genuine bug found by the third review pass: since transferring ownership from user A to user B necessarily involves two logical writes (demote A, promote B), doing this as two separate ad-hoc statements — as an admin editing permissions directly might naturally do — creates a moment where the constraint would see either zero owners or two owners, depending on statement order. Making the constraint deferrable (checked at transaction commit, not after each statement) and centralizing the operation into one dedicated "transfer ownership" routine closes this gap. **No code path — including admin actions — should ever `UPDATE` / `DELETE` / `INSERT` PERMISSIONS rows to change ownership except through this single routine.**
- **Deleting a user must reassign their owned items, not cascade-delete the permission grants.** *(Pass 2.)* A naive `ON DELETE CASCADE` from USERS to PERMISSIONS would silently orphan every file/folder the deleted user owned (leaving them with zero owners, violating the very invariant the constraint above exists to protect). Account deletion needs an explicit "reassign ownership" or "transfer to admin" step before the user row itself is removed.

3.7 EFFECTIVE_PERMISSIONS (cache)

(Added in Pass 2.) A denormalized, precomputed cache of each user's *resolved* access level on each item, accounting for folder-level sharing that cascades down to contents without literally copying a PERMISSIONS row for every file in a shared folder.

Field	Type	Notes
`item_type, item_id, user_id`	composite PK	
`role`	enum	The resolved/highest applicable role.
`source_permission_id`	FK → PERMISSIONS	Which grant this resolution came from — useful for explaining "why do I have access to this" in a UI.
`recomputed_at`	timestamp	

Why this table exists: the underlying design choice — sharing a folder does *not* copy a PERMISSIONS row down to every file inside it, to keep the PERMISSIONS table small and avoid rewriting thousands of rows every time someone shares a large folder — means that "can user X read file Y" naturally requires walking up the folder ancestor chain checking for a grant at each level. Doing that walk synchronously on

every single file access is expensive at real folder depths. This cache exists purely to make that check fast.

Rebuild mechanism (clarification): "a background job" rebuilds this cache — concretely, this means every PERMISSIONS-affecting write (share, unshare/revoke, ownership transfer) and every FOLDERS/FILES move, after committing its own transaction, enqueues a small unit of work (e.g., "recompute effective permissions for subtree rooted at item X") onto an application-level job queue, which a worker process picks up and processes by walking the affected subtree and upserting EFFECTIVE_PERMISSIONS rows. This document does not prescribe a specific queue technology (the same durable-queue *pattern* already used for DELETION_QUEUE and THUMBNAIL_QUEUE is the natural fit, and could plausibly be the same underlying mechanism); the requirement is only that the rebuild is asynchronous, decoupled from the triggering transaction, and retryable — not that it uses any particular tool.

Constraints and behavioral rules — this is the most heavily revised entity in the whole document:

- **(Pass 2, initial version)** The cache is rebuilt by a background job whenever a share, unshare, or move event occurs.
- **(Pass 2 review's own follow-up finding)** The initial description didn't actually specify that revoke/unshare is one of the triggering events, nor that a rebuild triggered by a *folder*-level change must cascade the rebuild down to every descendant file and folder, not just the one item that changed. Both are now explicit requirements: **revoking access to a folder must cascade the cache rebuild to its entire subtree**, and **moving a file must trigger a rebuild reflecting its new location**, not just leave the file processed once at its original path.
- **(Pass 2 — the single most important correctness fix in the whole schema)** Because the rebuild is asynchronous, there is an unavoidable staleness window between a permission change committing and the cache reflecting it. For a sharing feature, this matters enormously: a user who has just been unshared from a folder could, in the naive design, retain read access to files inside it for as long as the rebuild job takes to run. **The fix: this cache must never be treated as the source of truth for an access-control decision.** It is a read-path performance optimization only. Every actual permission check (can this user read/write this file right now) must be able to fall back to a synchronous, on-demand recomputation from the real PERMISSIONS table if the cache is missing, stale, or the check is for something security-sensitive. The cache makes the common case fast; it must never be the only thing standing between a user and unauthorized access.
- **(Pass 3)** The admin bypass (§4) must route through this *same* access-check function/code path — with `role = admin` short-circuiting the check inside it — rather than being a separate, hand-rolled check on individual endpoints. This avoids a class of bug where a new admin-only feature forgets to implement the bypass correctly, or implements it inconsistently with how regular access checks work.
- **(Pass 4)** FAVORITES visibility (§3.11) must also resolve through this same check, not just through `deleted_at IS NULL`. A favorited item that still exists but that the user has since lost access to (permission revoked, share link expired) should not silently keep surfacing in their favorites list.
- **(Pass 5 — final-pass finding)** `transfer_ownership()` (§3.6) must also be added to the list of events that trigger an EFFECTIVE_PERMISSIONS rebuild, alongside share/unshare/move. It was omitted from that list because it was designed and reviewed separately (as a fix for the ownership-uniqueness race, §3.6) and never explicitly cross-referenced against the cache-invalidation rule established here. Functionally this gap is low-severity — because the cache is never trusted as a

source of truth and every access check has a synchronous fallback (the rule directly above), a stale cache row here cannot itself grant incorrect access — but it does leave EFFECTIVE_PERMISSIONS rows referencing a `source_permission_id` that ownership transfer just invalidated, contradicting the "cascades to descendants" invariant this table is supposed to uphold. **Fix: `transfer_ownership()` must either invoke the same rebuild routine used for share/unshare/move, or be added explicitly to the trigger list wherever that routine is wired up.**

3.8 SHARE_LINKS

Public, tokenized share links — access without requiring a user account.

Field	Type	Notes
`id`	UUID, PK	
`item_type`, `item_id`	polymorphic	Same pattern as PERMISSIONS.
`token`	string, unique, indexed	The secret in the shareable URL.
`role`	enum: `viewer`, `editor`	Deliberately excludes `owner` — ownership (§3.6) is a stronger relationship than any share can confer, since it carries responsibilities (exactly one per item, transferable only via `transfer_ownership()`) that don't make sense for an anonymous link recipient with no account.
`password_hash`	string, nullable	Optional extra protection on top of the token.
`expires_at`	timestamp, nullable	
`created_by`	FK → USERS	
`created_at`	timestamp	

No structural issues were found with this table across any review pass; it's included here for completeness of the full picture.

3.9 ACTIVITY_LOG

An append-only record of actions taken in the system.

Field	Type	Notes
`id`	UUID, PK	
`actor_id`	FK → USERS	**(Pass 3, renamed/split from a single `user_id`)** Who performed the action.

Field	Type	Notes
`subject_user_id`	FK → USERS	**(Pass 3, new)** Whose account or item was affected by the action. Equal to `actor_id` for ordinary self-service actions (a user uploading their own file); diverges from it when an admin acts on another user's behalf or data.
`item_type`, `item_id`	polymorphic, indexed	What the action was performed on.
`action`	enum	The complete set of loggable actions: `upload`, `download`, `rename`, `move`, `delete` (soft), `restore`, `purge` (hard delete completion), `share`, `unshare`, `create_share_link`, `revoke_share_link`, `favorite`, `unfavorite`, `tag`, `untag`, `quota_change`, `permission_change`, `ownership_transfer`. This list is closed for the purposes of this design — any new user-facing action added later should extend this enum explicitly rather than relying on `metadata` alone to carry it.
`metadata`	jsonb	Free-form extra context (e.g., old/new values for a rename or quota change).
`created_at`	timestamp, indexed	

Why the actor/subject split matters *(Pass 3 finding)*: with a single `user_id` column, there was no way to correctly log an admin action. Logging it as the admin loses the record of *whose* account was affected (an audit trail that can't answer "show me every change made to user X's account" is not much of an audit trail). Logging it as the affected user falsely attributes the admin's action to the victim, defeating the entire purpose of an admin audit trail. The two-column design resolves this cleanly.

Confirmed by the system owner: this actor/subject split is sufficient for the admin audit trail as-is — no additional reason/comment field or stronger audit guarantee (e.g., requiring justification text on quota changes or permission overrides) is needed beyond what `actor_id`, `subject_user_id`, `action`, and `metadata` already capture.

Also serves as the data source for admin "usage frequency" statistics *(admin feature requirement)* — computed via aggregation (count of actions grouped by item and/or action type over a time window), rather than a separate always-maintained counter table. This was a deliberate simplicity trade-off: it's slower on a very large history than a dedicated counter table would be, but avoids introducing yet another denormalized cache that needs its own invalidation story.

3.10 DELETION_QUEUE

(Added in Pass 1, substantially extended in Pass 2.) A durable work queue for asynchronous cleanup after a hard delete.

Field	Type	Notes
`id`	UUID, PK	
`item_type`, `item_id`	polymorphic	What's being deleted.
`storage_key`	string, nullable	The blob to remove from the object store.
`thumbnail_key`	string, nullable	**(Pass 4)** The thumbnail blob to remove alongside the main object, in the same worker pass.
`owner_id`	FK → USERS	**(Pass 2, added)** Needed so the worker knows whose quota to credit back.
`size_bytes`	bigint	**(Pass 2, added)** Needed for the same reason.
`status`	enum: `pending`, `processing`, `done`, `failed`	(`processing` **added in Pass 5** to support the idempotent-enqueue constraint below — a status of just `pending` / `done` / `failed` can't distinguish "not yet claimed by a worker" from "a worker has it right now," which the uniqueness guard needs to protect against both.)
`attempts`	integer	Retry counter.
`enqueued_at`, `processed_at`	timestamp	

`UNIQUE(item_id) WHERE status IN ('pending', 'processing')` *(Pass 5.)* Prevents the 30-day scheduled purge job and an explicit "delete now" action from both successfully enqueueing the same item — see §5.2 for the full scenario. A second enqueue attempt while one is already in flight becomes a no-op rather than a duplicate row.

Retry policy (clarification): a row moves `pending → processing` when a worker claims it, then to `done` on success or back to `pending` (with `attempts` incremented) on a retryable failure — e.g., a transient MinIO connection error. After a fixed maximum number of attempts (an application-level constant, not modeled as a schema field), the worker instead sets `status = 'failed'` and stops retrying automatically. A `failed` row is not silently dropped: it should surface on an admin-visible "failed deletions" view for manual investigation, since a permanently stuck deletion means a blob or database row is not being cleaned up as expected. The same retry pattern (bounded automatic retries, then a terminal `failed` state requiring attention) applies identically to THUMBNAIL_QUEUE (§3.13), with the difference that a failed thumbnail job is lower-stakes — it can reasonably just leave `FILES.thumbnail_status = 'failed'` with no further escalation, since a missing preview image is a cosmetic gap, not a resource leak or security concern.

Why this table exists at all *(Pass 1 finding)*: the original design deleted a FILES row and expected the application to separately delete the corresponding object from the blob store in the same request. This is unsafe — if the app crashes or errors between the two operations, the blob is either orphaned forever (DB row gone, object still consuming storage) or the DB row survives pointing at a now-missing object. A durable queue with retry lets deletion be safely decoupled and recovered from failure.

Behavioral rules:

- **The worker's job, in order: delete the object from the blob store, then purge the database row, and cascade-clean any PERMISSIONS, SHARE_LINKS, and ACTIVITY_LOG rows that reference the deleted item.** *(Pass 1.)* Because PERMISSIONS/SHARE_LINKS/ACTIVITY_LOG reference items polymorphically with no real foreign key, nothing else in the database will clean these up automatically — this queue is the only place that cascade happens.
- **When a folder is purged, every descendant file must be enumerated individually and given its own queue row — not just one row for the folder itself.** *(Pass 2 — this closed a real orphaned-blob bug.)* The original round-1 description of this worker only mentioned purging "the item," which for a folder would delete the folder's own metadata but leave every child file's blob permanently orphaned in the object store, since nothing had ever enqueued their storage keys individually.
- **On successful completion, the worker must atomically decrement the owner's `storage_used_bytes` by `size_bytes`, in the same transaction as marking the queue row done.** *(Pass 2 — this closed a real "quota never frees up" bug.)* The original design updated `storage_used_bytes` on upload but nothing in the deletion path ever credited it back, meaning a user's available quota would only ever shrink over the lifetime of the account.
- **System-wide storage statistics have a caveat.** *(Pass 3.)* `SUM(storage_used_bytes)` across all users is the cheapest way to compute total storage used, but it does not account for bytes tied up in in-flight UPLOAD_SESSIONS or DELETION_QUEUE rows that are still `pending`. Any admin-facing "total storage used" figure should be labeled as a settled/point-in-time approximation, not represented as an exact real-time number.
- **The cascade cleanup responsibility now also covers FAVORITES and ITEM_TAGS.** *(Pass 4.)* When these two entities were added, the worker's existing cascade step (previously "clean up PERMISSIONS, SHARE_LINKS, ACTIVITY_LOG") was extended to also delete any FAVORITES or ITEM_TAGS rows referencing the purged item — otherwise both would silently accumulate references to items that no longer exist, the same class of bug the original DELETION_QUEUE design was built to prevent for the first three tables.
- **THUMBNAIL_QUEUE is deliberately *not* part of this manual cascade list, and this is correct — not an oversight.** *(Pass 5 — clarified during final review, since the omission was ambiguous without this note.)* Every table in the manual cascade list (PERMISSIONS, SHARE_LINKS, ACTIVITY_LOG, FAVORITES, ITEM_TAGS) uses the polymorphic `(item_type, item_id)` pattern with no real database foreign key, which is exactly *why* they need this worker to clean them up explicitly — nothing else will. THUMBNAIL_QUEUE is different: it has a real `file_id` foreign key back to FILES declared `ON DELETE CASCADE` (§3.13), so the database itself automatically removes any THUMBNAIL_QUEUE row the moment DELETION_QUEUE's worker purges the corresponding FILES row — no manual cleanup step is needed or should be added.

3.11 FAVORITES

(Added in Pass 4.) Lets a user mark a file or folder as a personal favorite/starred item, independent of who owns it.

Field	Type	Notes
`user_id`	FK → USERS	
`item_type`, `item_id`	polymorphic	Same pattern as PERMISSIONS.
`created_at`	timestamp	

Primary key: `(user_id, item_type, item_id)` — this composite key is itself the uniqueness guarantee; a user cannot favorite the same item twice, and no separate surrogate `id` is needed.

Constraints and behavioral rules:

- **A secondary index on `(item_type, item_id)` is required, separate from the primary key.** *(Pass 4 review finding.)* Because `item_type` / `item_id` are the *trailing* columns of the primary key (not the leading ones), the PK alone does not efficiently serve the reverse lookup "what favorited this item" — which DELETION_QUEUE's cascade cleanup needs on every purge. Without this explicit secondary index, that cleanup step degrades to a full table scan.
- **Favoriting a folder does not require re-validating access on every subsequent view — visibility is resolved lazily, not enforced eagerly.** *(Pass 4 — this was the most significant finding for this entity.)* If a user favorites a folder and later loses access to it (their PERMISSIONS grant is revoked, or a SHARE_LINK expires), nothing about that revocation event touches FAVORITES — it isn't a DELETION_QUEUE trigger, since the item itself hasn't been deleted, only the user's visibility into it has changed. The correct design is **not** to try to eagerly detect every access-loss event (that would require hanging a new side effect off of every permission-revocation code path, a disproportionate amount of new complexity for a "starred items" feature). Instead: when rendering a user's favorites list, resolve each entry through the same visibility check as everywhere else — `deleted_at IS NULL` **and** an EFFECTIVE_PERMISSIONS check (§3.7) — and if a row fails that check, delete it from FAVORITES at that point (a lazy prune-on-read), rather than merely hiding it in the UI. This keeps FAVORITES consistent without adding a new invalidation trigger to the permissions system.

3.12 TAGS and ITEM_TAGS

(Added in Pass 4.) User-defined labels that can be applied to files and folders. Two tables, because a tag is a reusable named object distinct from any single application of it.

TAGS — the tag definitions themselves:

Field	Type	Notes
`id`	UUID, PK	
`owner_id`	FK → USERS	The tag's creator.
`name`	string	
`color`	string, nullable	For UI display.

Field	Type	Notes
`created_at`	timestamp	

- **`UNIQUE(owner_id, name)`** — tags are scoped **per-user**, not global. **Confirmed by the system owner as the intended design, not just a placeholder default.** This sidesteps a much bigger design question: if tags were shared/global, two different users' "Work" tags would either collide (forcing a decision about whether they mean the same thing) or need some other disambiguation mechanism. Per-user scoping avoids that entirely at the cost of users not being able to share a tag taxonomy — an accepted trade-off, consistent with team/group accounts being explicitly out of scope (§1.2).
- **On user deletion, TAGS rows are cascade-deleted — they are NOT reassigned to another user.** *(Pass 4 — this was an explicit correction of an initially-proposed design.)* The initial draft of this feature assumed TAGS should follow the same "reassign on user deletion" pattern established for PERMISSIONS ownership (§3.6). Review found this analogy doesn't hold: the PERMISSIONS reassignment rule exists because a file or folder is a shared resource that persists independently of any one user's relationship to it, and *must* always have exactly one owner (a database constraint enforces this). A tag has no such independent existence and no other stakeholders — it is a personal label with meaning only to its creator. Reassigning someone's "Work" tag to an arbitrary remaining user would be semantically meaningless (whose tag is it now, and why?). The correct rule is simply: deleting a user cascade-deletes their TAGS, which in turn cascade-deletes the associated ITEM_TAGS rows (below) — the tagged items themselves are entirely unaffected; they simply lose that tag.

ITEM_TAGS — the application of a tag to a specific file or folder:

Field	Type	Notes
`tag_id`	FK → TAGS, `ON DELETE CASCADE`	
`item_type`, `item_id`	polymorphic	Same pattern as PERMISSIONS.
`created_at`	timestamp	

- **Primary key:** `(tag_id, item_type, item_id)`.
- **A secondary index on `(item_type, item_id)` is required** — same reasoning as FAVORITES above: the PK's column order doesn't serve the "what tags does this item have" / DELETION_QUEUE reverse-lookup direction.
- **Writing to ITEM_TAGS requires the acting user to hold `editor` role or above on the target item, via EFFECTIVE_PERMISSIONS — independent of who owns the tag being applied.** *(Pass 4 — this closed a real gap, not a hypothetical one.)* The initial draft specified `TAGS.owner_id` (who created the tag) but never specified any permission check on the *item* being tagged. As drafted, this would have allowed a user with only `viewer` access to a shared file to tag it — despite having no write access to the file itself — which is inconsistent with every other write operation in this schema. The fix makes tagging follow the same write-permission rule as renaming, moving, or deleting: `editor` or above.

3.13 THUMBNAIL_QUEUE

(Added in Pass 4.) An asynchronous work queue for generating preview thumbnails, structurally identical in spirit to DELETION_QUEUE (§3.10) — decoupling a slow, failure-prone operation (image/video/PDF rendering) from the request that triggered it.

Field	Type	Notes
`id`	UUID, PK	
`file_id`	FK → FILES, `ON DELETE CASCADE`	
`status`	enum: `pending`, `done`, `failed`	
`attempts`	integer	Retry counter.
`enqueued_at`, `processed_at`	timestamp	

Constraints and behavioral rules:

- **A row is only enqueued for `image`, `video`, and `pdf` `mime_type` values** — there's no meaningful thumbnail to generate for, e.g., a `.zip` or a plain text file.
- **`UNIQUE(file_id) WHERE status = 'pending'`** *(Pass 4 — this closed a real concurrency bug.)* Nothing in the initial draft prevented two concurrent upload-completion handlers from both enqueueing a thumbnail job for the same file — for example, a retried request or a race between two app server instances. Without this partial unique index, that produces duplicate queue rows, duplicated worker effort, and a possible last-writer-wins clobber of `thumbnail_key` if both workers happen to run close together. With it, a second enqueue attempt while one is already pending is a no-op (constraint conflict, handled as "already queued") rather than a duplicate.
- **`INDEX(status, enqueued_at)`** for the worker to efficiently find and process pending rows in order.
- **The worker must re-check `FILES.deleted_at IS NULL` immediately before writing the generated thumbnail blob to the object store — not just when the job was first enqueued.** *(Pass 4 — this closed a real orphaned-blob bug, found by explicitly tracing the race between this queue and DELETION_QUEUE.)* Concrete failure scenario without this check: a file is uploaded, a thumbnail job is enqueued, and the file is deleted before the worker gets to it. The worker, still holding the original `file_id` and unaware of the deletion, can finish generating and writing a thumbnail blob to MinIO *after* the file has already been purged — producing a real, permanently orphaned object with no reference anywhere pointing back to it (since the FILES row that would have carried `thumbnail_key` is already gone). Re-checking immediately before the write — combined with `ON DELETE CASCADE` on `file_id` so the queue row itself vanishes when the FILES row is hard-deleted — closes this. This works together with DELETION_QUEUE's `thumbnail_key` column (§3.10) to also purge whatever thumbnail *did* successfully complete before a subsequent deletion.

---

4. The Admin / Super-User Feature

This section consolidates every design decision specific to the administrative role, since it was added after the base schema and reviewed separately.

4.1 Requirements

An admin can:

- View all files and folders across all users, regardless of ownership or sharing.
- See per-user and system-wide storage statistics (used, remaining/quota).
- See "usage frequency" — how often files are accessed, derived from ACTIVITY_LOG.
- Alter other users' permissions (including reassigning ownership).
- Change other users' storage quotas.

4.2 Design decisions

Decision	Rationale
Admin status is a `role` column directly on USERS, not a separate `ADMIN_ROLES` grant table.	The requirement is a simple binary (is/isn't an admin), not a system needing multiple admin tiers or revocable/audited admin grants. A column is the simplest thing that satisfies the requirement; a join table would be premature generality.
Admin's ability to view any file bypasses PERMISSIONS/EFFECTIVE_PERMISSIONS checks entirely at the application layer — there are no database rows granting admin access to every item.	Avoids having to backfill a PERMISSIONS row per item per admin (which would also need to stay in sync as new items are created). The bypass logic lives in one shared access-check function (see §3.7) so it's applied consistently rather than re-implemented per endpoint.
"Usage frequency" is computed by aggregating ACTIVITY_LOG rather than maintained as a live counter table.	Simplicity — avoids introducing another denormalized cache with its own invalidation rules, at the cost of being slower to compute over very large histories. Acceptable trade-off for a self-hosted deployment's scale.

4.3 Known, accepted risks of this design

These were surfaced during review and are being documented as **conscious trade-offs**, not silently accepted gaps:

- **Pure application-layer bypass has no database-level backstop.** If a new API endpoint, background job, or an ad-hoc SQL script forgets to apply the admin/permission check, that code path has *no* access control at all — not "admin gets extra access," but "anyone who can reach it gets full access," since there's no PERMISSIONS row to fall back to either. This is judged an acceptable trade-off specifically because of the single-server, small-trusted-admin-surface deployment shape (§1.3). Mitigations: route every access check through one shared authorization function (§3.7), and keep any direct/raw-SQL admin database access on a separate credential from the application's own runtime

database connection, so that kind of access is a deliberate, distinguishable action rather than something indistinguishable from normal app traffic.

- `storage_key` **plus direct object-store access bypasses the entire permission model.** Nothing about PERMISSIONS or EFFECTIVE_PERMISSIONS constrains the object store itself — anyone with direct MinIO bucket access, or who obtains a `storage_key` through some other means, can fetch the raw bytes with no regard for who's supposed to have access. This is extremely common and generally accepted for self-hosted single-tenant deployments, but it's a stated assumption, not an oversight: it requires that MinIO credentials are scoped only to the application service (not shared with, e.g., an admin's personal tooling) and that the bucket policy denies anonymous/public reads (see §3.4).
- **"Total storage used" system-wide stats are an approximation**, not an exact real-time figure, for the reasons described in §3.10.

---

5. Cross-Cutting Design Patterns

5.1 The polymorphic `item_type` / `item_id` pattern

PERMISSIONS, SHARE_LINKS, ACTIVITY_LOG, DELETION_QUEUE, FAVORITES, and ITEM_TAGS all need to reference "either a file or a folder." Two alternatives were considered and rejected:

- **A single unified `ITEMS` table** with a `type` discriminator, merging FILES and FOLDERS into one entity. This would give a clean foreign key target for the polymorphic tables, but was rejected to keep file-specific fields (size, checksum, storage_key, mime_type) and folder-specific fields (materialized_path) from living awkwardly in one table with a lot of nullable columns, and to keep queries for "list files in this folder" simple rather than needing a type filter on every query.
- **Two separate junction tables per relationship** (e.g. `FILE_PERMISSIONS` and `FOLDER_PERMISSIONS`), doubling the number of tables for every relationship that needs to target either type.

The chosen approach — a `(item_type, item_id)` pair with no real database-level foreign key (Postgres cannot express a foreign key against "one of two tables") — is the pragmatic middle ground. Its costs, which are accepted as a deliberate trade-off, are:

- No referential integrity from the database itself; an orphaned `item_id` pointing at a deleted item is possible if the DELETION_QUEUE cascade cleanup (§3.10) is ever skipped or fails.
- Every query joining against "the actual item" needs an application-level `CASE` / `UNION` to resolve into FILES or FOLDERS, rather than a simple join.
- Composite indexes on `(item_type, item_id)` are mandatory on every table using this pattern (PERMISSIONS, SHARE_LINKS, ACTIVITY_LOG, DELETION_QUEUE, FAVORITES, ITEM_TAGS) — without them, resolving "everything shared with this item" requires a full table scan. Note that for FAVORITES and ITEM_TAGS specifically, this index must be a distinct *secondary* index rather than relying on the primary key, since `item_type` / `item_id` are the trailing columns of those tables' composite primary keys (§3.11, §3.12) and therefore don't serve this lookup direction on their own — a subtlety the polymorphic pattern makes easy to miss when adding a new table that uses it.

5.2 Soft delete and trash

FILES and FOLDERS both use a `deleted_at` timestamp rather than a separate TRASH table. A null value means the item is live; a non-null value means it's in the trash. This was a deliberate simplification — a separate trash table would need to duplicate most of the same fields and be kept in sync, for no real benefit over a nullable column with a partial index (`WHERE deleted_at IS NULL`) used throughout the schema to exclude trashed items from normal queries.

Two-stage deletion lifecycle:

1. **Soft delete (trash):** setting `deleted_at` on a folder must cascade to its entire subtree in the same transaction (a recursive operation, not a single-row update) — otherwise child items remain "live" (`deleted_at IS NULL`) while being unreachable through a trashed parent, an inconsistent state.
2. **Hard delete (purge):** triggered one of two ways — **(a)** a scheduled job that automatically purges anything that has sat in trash for **30 days** (the confirmed retention window), or **(b)** the user explicitly choosing to permanently delete an item immediately, bypassing the retention window. **Both paths are required — the retention window is a default grace period, not the only way to purge; the user must always have the option to force an immediate permanent delete rather than wait out the 30 days.** Either way, this is where DELETION_QUEUE (§3.10) takes over — enumerating the full subtree, enqueuing blob deletion and cascade cleanup for every descendant, and crediting quota back on completion. - **(Pass 5 finding)** The scheduled 30-day job should enumerate and enqueue only top-level trashed roots (items whose `deleted_at` is set but which are not themselves a descendant of another already-trashed item) — it should **not** re-derive DELETION_QUEUE's descendant-enumeration logic; that walk already happens once, inside the queue's own worker, regardless of which path triggered the enqueue. - **(Pass 5 finding — real race, now closed)** Because both the scheduled job and the explicit "delete now" action can target the same item, they can race each other — both attempting to enqueue the same item around the same time, which without a guard could cause DELETION_QUEUE's worker to process the same subtree twice (double-decrementing `storage_used_bytes` , or erroring on an already-purged row/object). **Fix: DELETION_QUEUE needs `UNIQUE(item_id) WHERE status IN ('pending', 'processing')` , so a second enqueue attempt** for an item already in flight is a no-op (`ON CONFLICT DO NOTHING`) rather than a duplicate. - **(Pass 5 finding)** Because purge processing is asynchronous, there's an inherent — and acceptable — brief window between an item being enqueued for permanent deletion and a worker actually completing that purge, during which the UI could still show it as "in trash" if it derives that state naively from `deleted_at` alone. **The restore-eligibility check must also test for the absence of a pending/processing DELETION_QUEUE row for that item, not just `deleted_at IS NOT NULL` —** otherwise a user could click "restore" on an item that is already mid-purge.

5.3 Denormalized counters and caches — and why none of them are trusted blindly

This schema deliberately uses three pieces of denormalized/derived state for performance: `storage_used_bytes` (USERS), `materialized_path` (FOLDERS), and EFFECTIVE_PERMISSIONS. Each exists purely because computing the equivalent value live (via `SUM()` , a recursive ancestor query, or a permission tree-walk) would be too slow to do on every relevant request.

The recurring lesson across all three review passes is the same: **a derived value is only safe if there's a precise answer to "what keeps this in sync, and what happens if it's temporarily wrong."** For each of the three:

- `storage_used_bytes` is kept in sync by requiring every mutation (upload, deletion-queue completion) to update it atomically, in the same transaction, using a conditional update rather than a read-then-write (to avoid a race where two concurrent uploads both check quota against a stale value and both succeed, together exceeding it).
- `materialized_path` is kept in sync by requiring a full-subtree rewrite inside the same transaction as any folder move.
- EFFECTIVE_PERMISSIONS is *not* fully trusted — it's explicitly designated a read-optimization with a synchronous-recompute fallback, precisely because "what keeps this in sync" (an async background job) cannot give an airtight answer to "what happens if it's temporarily wrong" for something as sensitive as an access-control decision.

No periodic reconciliation/repair job (recomputing all derived state from source tables to catch drift) has been designed yet; this is a reasonable follow-up but wasn't specified in detail. It's flagged here as an intentionally deferred item rather than a review finding.

5.4 Indexing

Indexes required by the design, consolidated from every entity above (these are conceptual requirements — see §3 for context on why each exists):

- `FOLDERS(parent_folder_id)`, `FOLDERS(materialized_path)` (prefix-searchable)
- `FILES(folder_id)`, `FILES(owner_id)`
- `FOLDERS(owner_id)`
- Partial indexes `WHERE deleted_at IS NULL` on both FILES and FOLDERS — nearly every query filters out trashed items, and the partial unique-name constraints (§3.3, §3.4) depend on this filter too.
- `PERMISSIONS(item_type, item_id)`, `PERMISSIONS(user_id)`
- `SHARE_LINKS(token)` (should be backed by an index, not merely a uniqueness constraint, since it's looked up on every public link access), `SHARE_LINKS(item_type, item_id)`
- `ACTIVITY_LOG(item_type, item_id)`, `ACTIVITY_LOG(actor_id, created_at)`, `ACTIVITY_LOG(subject_user_id, created_at)`
- `EFFECTIVE_PERMISSIONS` primary key `(item_type, item_id, user_id)` doubles as its own lookup index.
- `DELETION_QUEUE(status)` — the worker needs to efficiently find pending rows.
- `FAVORITES(item_type, item_id)` — secondary index, required for the DELETION_QUEUE reverse-lookup cascade (§5.1).
- `TAGS(owner_id)` — for "list this user's tags."
- `ITEM_TAGS(item_type, item_id)` — secondary index, same reverse-lookup reasoning as FAVORITES; `ITEM_TAGS(tag_id)` for "show all items with this tag."
- `THUMBNAIL_QUEUE(file_id) WHERE status = 'pending'` — the partial unique index doubles as the concurrency guard (§3.13) and a lookup index.

- `THUMBNAIL_QUEUE(status, enqueued_at)` — for worker polling, same pattern as DELETION_QUEUE.
- `SESSIONS(refresh_token_hash)` — already required by its `UNIQUE` constraint, but called out explicitly here since it's looked up on every authenticated request and must stay fast.
- `UPLOAD_SESSIONS(user_id)`, `UPLOAD_SESSIONS(expires_at) WHERE status = 'pending'` — needed for the abandoned-upload cleanup job referenced in §3.5, which must efficiently find expired-but-still-pending sessions.

---

6. Summary of All Fixes by Review Pass

For quick reference — every change made, grouped by which pass introduced it.

Pass 1 (initial hardening): - Unique partial index on sibling names for both FILES and FOLDERS. - DELETION_QUEUE introduced, to decouple blob deletion from DB row deletion and make it retryable. - Cascade-delete semantics defined: folder trash/purge must recursively affect the whole subtree; nothing cascades automatically for the polymorphic tables, so DELETION_QUEUE must explicitly clean up PERMISSIONS/SHARE_LINKS/ACTIVITY_LOG. - Cycle-prevention trigger on folder moves (first version — later found incomplete in Pass 2). - Atomic, conditional quota update pattern instead of read-then-write, to close a race condition. - SESSIONS, UPLOAD_SESSIONS added — nothing previously modeled authentication or multipart uploads. - Full indexing pass.

Pass 2 (verifying Pass 1's fixes actually work): - `materialized_path` added to FOLDERS, with the requirement that a move rewrites the *entire subtree's* paths in one transaction. - Cycle-check trigger upgraded from "checks direct parent" to "must walk the full ancestor chain." - EFFECTIVE_PERMISSIONS cache introduced, explicitly designed as a read-optimization only, never a source of truth — access checks always retain a synchronous fallback. - File moves invalidate inherited EFFECTIVE_PERMISSIONS from the old folder. - Exactly-one-owner constraint added to PERMISSIONS. - User deletion must reassign owned items rather than cascade-delete grants. - DELETION_QUEUE's folder-purge behavior corrected to enumerate every descendant file individually, closing an orphaned-blob bug. - DELETION_QUEUE worker wired to credit storage back to the owner's quota on completion, closing a "quota never frees up" bug.

Pass 3 (reviewing the admin feature's consequences): - Admin role added as a `role` column on USERS. - Quota `CHECK` constraint scoped to fire only on increases, so admin can legitimately lower a user's quota below current usage. - Owner-uniqueness constraint made `DEFERRABLE INITIALLY DEFERRED`; ownership transfer centralized into a single atomic operation, closing a race condition specific to admin-driven permission changes. - Admin's access bypass explicitly routed through the same shared access-check function as regular users, rather than a separately hand-rolled check. - ACTIVITY_LOG split into `actor_id` / `subject_user_id` to correctly audit admin actions taken on other users' behalf. - `storage_key` non-guessability and MinIO bucket-policy requirements documented, since admin's "view all files" capability made it worth stating explicitly that the object store itself is not access-controlled. - Cycle-detection trigger's concurrency gap flagged as the single highest-risk item still left as a design note rather than a fully specified mechanism. - System-wide storage stats caveat documented (excludes in-flight uploads/pending deletions).

Owner decisions (post-Pass-4, resolving remaining open items): - Cycle-check locking strategy decided: **advisory lock on the subtree root** before the ancestor-walk check, over `SERIALIZABLE` +retry or row-level `FOR UPDATE` locking — chosen for implementation simplicity at this deployment's expected concurrency level. This closes the single highest-risk open item flagged across three review passes. - Tag scope confirmed as **per-user** (not global/shared) — consistent with team/group accounts being out of scope. - Team/group accounts confirmed **out of scope** — sharing remains exclusively individual-user-to-user or via public SHARE_LINKS. - Trash retention window set to **30 days**, with an explicit requirement that users can **permanently delete immediately**, bypassing the retention window, rather than the 30-day window being the only path to a hard purge. - Admin audit trail (the `actor_id` / `subject_user_id` split on ACTIVITY_LOG) confirmed **sufficient as designed** — no additional reason/justification field required. - Deduplication **held, not designed** — explicitly deferred rather than built now, revisit later if needed. The `checksum` field on FILES remains in place as a low-cost stepping stone for whenever that design work happens.

Pass 4 (adding favorites, tags, and thumbnails; reviewing them against the established patterns): - FAVORITES table added; visibility resolved through the same EFFECTIVE_PERMISSIONS check as everything else, with lost-access favorites lazily pruned on read rather than eagerly invalidated. - FAVORITES given a required secondary index on `(item_type, item_id)`, since its primary key's column order doesn't serve the DELETION_QUEUE reverse lookup. - TAGS and ITEM_TAGS added; tags scoped per-user to avoid a cross-user tag-collision design question. - TAGS given cascade-delete (not reassignment) semantics on user deletion — an explicit correction after review found the PERMISSIONS ownership-reassignment pattern didn't actually apply to a tag, which (unlike a file or folder) has no stakeholders beyond its creator. - ITEM_TAGS write path given an explicit `editor`-or-above permission check, independent of tag ownership — closing a gap where any user with any access level could have tagged an item they couldn't otherwise modify. - FILES extended with `thumbnail_status` / `thumbnail_key`; THUMBNAIL_QUEUE added, modeled directly on the DELETION_QUEUE pattern. - `thumbnail_key` given the same non-guessability / bucket-policy requirement as `storage_key`. - A concurrency bug closed with `UNIQUE(file_id) WHERE status = 'pending'` on THUMBNAIL_QUEUE, preventing duplicate concurrent thumbnail jobs for the same file. - A genuine race condition between THUMBNAIL_QUEUE and DELETION_QUEUE identified and closed: a file deleted while its thumbnail job is still pending could previously result in a thumbnail blob being written to the object store *after* the file itself was purged, with nothing ever referencing it again. Fixed via `ON DELETE CASCADE` on the queue row plus a mandatory re-check of `FILES.deleted_at` immediately before the worker writes the blob. - DELETION_QUEUE's cascade-cleanup responsibility extended to also cover FAVORITES, ITEM_TAGS, and the `thumbnail_key` blob.

Pass 5 / final (verifying the owner's six closing decisions): - Advisory lock corrected from the plain session-scoped variant to `pg_advisory_xact_lock` — the originally decided approach would have leaked or misbehaved under transaction-mode connection pooling (e.g., PgBouncer), since session locks are tied to the physical connection, not the transaction. - Cycle-check locking scope corrected: locking only the moved node's own subtree root does not prevent two folders being moved into each other's subtrees simultaneously. Fixed by locking both the moved node's subtree root and the target parent's ancestry root, in a canonical (sorted) order. - DELETION_QUEUE given `UNIQUE(item_id) WHERE status IN ('pending', 'processing')` (and a new `processing` status value) to make enqueueing idempotent — closing a real race between the 30-day scheduled purge job and the explicit "delete now" bypass both targeting the same item. - Restore-eligibility check requirement

added: must test for an in-flight DELETION_QUEUE row, not just `deleted_at`, so a user can't restore an item that's already mid-purge. - Team/group accounts being out of scope confirmed as introducing no hedging artifacts in the existing schema — `PERMISSIONS.user_id` as a single non-nullable grantee required no changes. - Deduplication deferral confirmed as fully neutral — nothing in the current design (in particular, `checksum` sitting unused today) needs to change to make future dedup adoption smoother; it would be purely additive.

Pass 5 addendum — final internal-consistency audit (checking the accumulated fixes against each other, not new functionality): - `transfer_ownership()` (§3.6) was missing from EFFECTIVE_PERMISSIONS' list of cache-rebuild trigger events — added, since ownership transfer is a PERMISSIONS write of the same category as share/unshare and was only omitted because it was designed and reviewed as a separate fix for a different problem (the ownership-uniqueness race) and never explicitly cross-referenced against the cache-invalidation rule. - THUMBNAIL_QUEUE's absence from DELETION_QUEUE's manual cascade list was confirmed correct (its `ON DELETE CASCADE` foreign key already handles this automatically) but was ambiguous as previously documented — a clarifying note was added so a reader doesn't mistake the omission for a gap. - Three other potential inconsistencies were explicitly checked and confirmed clean: THUMBNAIL_QUEUE does not need the same `processing` status added to DELETION_QUEUE, since it has no equivalent dual-trigger race; the advisory-lock scope (subtree root + target parent's ancestry root) and the materialized_path rewrite scope (full subtree of the moved node) were verified to compose correctly rather than having silently diverged across passes; and no two fixes from different passes were found to directly contradict each other anywhere in the final design.

---

7. Open Items for the Next Design Phase

The folder-move locking strategy, tag scope, team/group accounts, trash retention, admin audit sufficiency, and deduplication have all now been decided by the system owner and verified in Pass 5 (see the decisions blocks in §6) and are no longer open. What remains genuinely open — all implementation-level decisions, none requiring further owner input:

1. **The `transfer_ownership()` operation** needs to be fully specified (as a stored procedure/function or an application-transaction pattern) rather than just named.
2. **A periodic reconciliation/repair job** for the three denormalized values (`storage_used_bytes`, `materialized_path`, EFFECTIVE_PERMISSIONS) has not been designed — currently there is no detector for drift between derived state and source-of-truth tables if an event-driven update is ever missed.
3. **Rate limiting, and any admin-tier granularity beyond a single binary role**, were both explicitly out of scope for this pass.
4. **Thumbnail generation itself** (which library/service renders images, video frames, and PDF first-pages into a preview) is an implementation detail deliberately left unspecified here — this document only covers the queue/storage data model around it, not the rendering mechanism.
5. **The 30-day trash retention window's scheduled-purge mechanism** (e.g., a daily cron-style job that finds and enqueues everything past its retention deadline) needs to be specified as part of the

DELETION_QUEUE implementation, alongside the already-designed explicit "delete now" path — both now share the same idempotent-enqueue guard (§3.10, §5.2).

This document reflects the schema as of the sixth and final design iteration (v6), following five adversarial review passes and the resolution of every owner-level decision. It is intended as a complete conceptual reference — the next step is translating each entity, constraint, and behavioral rule described here into actual PostgreSQL DDL (`CREATE TABLE`, `CHECK`, trigger, and function definitions).